

Reporte 1 (01 - 05 de noviembre de 2026): Recreación del juego Pong en Lenguaje Ensamblador

Mariana Angélica Flores Vega

Elihú Alejandro Ibarra Hernández

Leonardo Roberto Piedras Fuente

Ingeniería en Software

CETYS Universidad Campus Internacional Ensenada

Fundamentos de Sistemas Operativos

Organización de Computadoras

Dr. Ramón Santana Fernández

Mtro. Antonio de Jesús García Chávez

05 de noviembre de 2026

Plan de Trabajo Semanal:

Proyecto en Ensamblador

Proyecto: Pong.

Equipo: Pingue-Pongue.

Periodo: 01 a 05 de noviembre de 2026.

Datos del Proyecto

Nombre del Proyecto	Recreación del juego Pong en Lenguaje Ensamblador
Curso y Profesor	• Organización de Computadoras • Mtro. Antonio de Jesús García Chávez
Objetivo General	El objetivo general del presente proyecto es recrear el videojuego Pong en su lógica escrita en lenguaje ensamblador x86, integrada en un proyecto de C++ en Visual Studio mediante ensamblador en línea (__asm). Como etapa previa de aprendizaje y experimentación, el juego se desarrolla primero en ensamblador 8086 de l6 bits sobre DOSBox, un emulador del entorno MS-DOS, tomando como referencia el curso "[MASM] 8086 Assembly - Pong" de Programming Dimension. Esta etapa permite estudiar la arquitectura x86 en su forma original (véanse registros, segmentos, interrupciones de BIOS y memoria de video) sin la complejidad adicional de integrar C++ con Windows. Además. produce una versión de referencia cuya lógica se migrará después a x86 de 32 bits, sustituyendo los servicios de DOS por equivalentes en C++.
Arquitectura Objetivo	x86: • **Desarrollo y Aprendizaje**: 8086 en modo real (16 bits) • **Versión Final**: x86 de 32 bits (Win32)

Ensamblador y Enlazador	• **Desarrollo y Aprendizaje**: MASM y LINK. • **Versión Final**: Ensamblador en línea de Microsoft Visual C++ (__asm).
Simulador o Entorno de Ejecución	• **Desarrollo y Aprendizaje**: DOSBox 0.74-3. • **Versión Final**: Windows con Visual Studio.
Depurador	• **Desarrollo y Aprendizaje**: Pruebas visuales en DOSBox. • **Versión Final**: Depurador de Visual Studio (registros y memoria).
Convención de Llamada Acordada	• **Desarrollo y Aprendizaje**: NEAR sin parámetros que operan sobre variables globales (CALL / RET). • **Versión Final**: cdecl con funciones sin parámetros sobre variables globales.
Repositorio	https://github.com/elihuibah/pong-assembly
Comando de Compilación	• **Desarrollo y Aprendizaje**: ○ masm/a pong.asm ○ link pong ○ pong • **Versión Final**: Compilación de la solución en configuración Debug	x86.
Fecha de Entrega Final	Jueves, 26 de noviembre de 2026

Equipo y Responsabilidades

Integrante	Rol	Módulos o rutinas a cargo	Contacto
Elihú Ibarra	A: Curso de Desarrollo y Aprendizaje, y Revisiones.	Código de desarrollo y aprendizaje en dosbox/, estructura del programa, modo de video y rutinas de dibujo y lógica.	elihuh@cetys.edu.mx

Integrante	Rol	Módulos o rutinas a cargo	Contacto
Leonardo Piedras	B: Port a Visual Studio.	Plataforma en C++ (ventana, pantalla virtual, teclado, temporizador) y traducción de rutinas a __asm.	leonardo.piedras@cetys.edu.mx
Mariana Flores	C: Tests y Presentaciones Semanales	Pruebas, bitácora, presentaciones semanales, marcador y pantalla de fin de juego.	flores.mariana@cetys.edu.mx

Nota. Los integrantes de los roles B y C acordaron invertir sus roles a lo largo del proyecto.

Hitos del Proyecto

Hito	Criterio de Terminado	Semana Objetivo	Responsable	Estado
I). Entorno y pelota dibujada	El programa ensambla, cambia a modo gráfico y dibuja la pelota en DOSBox.	01 - 05 de noviembre	Elihú	Completo
II). Pelota en movimiento	La pelota se mueve y rebota en las cuatro paredes sin salirse en DOSBox.	06 - 12 de noviembre	Elihú y Leonardo	Pendiente
III). Paletas controlables	Ambas paletas se dibujan y se mueven con el teclado en DOSBox.	06 - 19 de noviembre	Leonardo y Mariana	Pendiente
IV). Colisión y marcador	La pelota rebota en las paletas y el marcador suma puntos.	12 - 19 de noviembre	Todos	Pendiente
V). Versión de Visual Studio	El juego completo corre en Win32 con gráficos equivalentes a DOSBox.	19 - 26 de noviembre	Todos	Pendiente

Hito	Criterio de Terminado	Semana Objetivo	Responsable	Estado
VI). Entrega final	Código probado, documentado y presentado.	26 de noviembre	Todos	Pendiente

Resumen de Avance

Se actualiza al cierre de cada semana. Cumplimiento = objetivos completados / objetivos planeados.

Semana	Planeados	Completados	Parciales	Pendientes	Fallas abiertas	Cumplimiento
1	6	6	0	0	0	100%

Guía de Llenado

Estados de un Objetivo

Estado	Significado
Completo	Cumple todo el criterio de terminado y tiene evidencia.
Parcial	Hay avance verificable, pero falta parte del criterio.
Pendiente	No se avanzó de forma verificable.
Cancelado	Ya no se hará. Se explica el motivo en la Parte B.

Criterio Mínimo de Terminado para Código

- Ensambla y enlaza sin errores con el comando acordado.
- Pasa los casos de prueba definidos, incluidos los casos límite.
- Respeta la convención de llamada: registros preservados y pila balanceada al salir de cada rutina.
- Cada rutina documenta sus entradas, salidas y registros que modifica.
- Está en el repositorio con un commit identificable.

Categorías de Causa de Falla

Categoría	Ejemplos
Registros	Registro sobrescrito, registro no preservado, tamaño de registro incorrecto.
Pila	Push y pop desbalanceados, marco de pila incorrecto, alineación, dirección de retorno dañada.
Memoria	Modo de direccionamiento incorrecto, acceso fuera de límites, endianness, datos sin alinear.
Control de flujo	Salto condicional equivocado, bandera mal interpretada, condición de ciclo con error de uno.
Aritmética	Desbordamiento, confusión entre operaciones con signo y sin signo, división sin preparar registros.
Sistema	Llamada al sistema o interrupción con parámetros incorrectos, error de enlace, sección equivocada.
Herramientas	Diferencias de sintaxis o de versión, fallas del simulador, entorno distinto entre integrantes.
Planeación	Tarea subestimada, dependencia no prevista, requisito mal entendido.
Coordinación	Integración tardía, interfaz entre rutinas no acordada, falta de disponibilidad.

Semana 1: 01 al 05 de noviembre de 2026

Hito al que contribuye	I). Entorno y Pelota Dibujada
Responsable del reporte	Elihú Ibarra
Commit o etiqueta al iniciar	54c1c44
Commit o etiqueta al cerrar	764b485

Parte A. Plan

Meta de la Semana

Levantar el entorno de desarrollo así como establecer un repositorio organizado, y lograr que el programa cambie a modo gráfico y dibuje la pelota en pantalla.

Objetivos

ID	Objetivo	Archivo o rutina	Responsable	Criterio de terminado	Horas est.
S1-O2	Configurar DOSBox y MASM	dosbox.conf	Elihú	DOSBox monta la carpeta automáticamente y ensambla un programa de prueba.	1
S1-O2	Organizar el repositorio de GitHub	• .gitignore • README	Elihú	Estructura por rol en GitHub, sin archivos de compilación ni herramientas de terceros.	0.5
S1-O3	Estructura base del programa	PONG.ASM	Elihú	Segmentos de pila, datos y código definidos. El programa ensambla y termina sin errores.	1

ID	Objetivo	Archivo o rutina	Responsable	Criterio de terminado	Horas est.
S1-O4	Modo de video y fondo	PONG.ASM	Elihú	La pantalla cambia a modo gráfico con el fondo correcto.	1
S1-O5	Dibujar la pelota	DRAW_BALL	Elihú	Se dibuja un cuadrado de BALL_SIZE píxeles en BALL_X, BALL_Y.	1
S1-O6	Plan de trabajo y roles	docs/	Todos	Plan entregado y roles asignados.	2.5

Riesgos previstos

Riesgo	Objetivo afectado	Qué haremos si ocurre
Curva de aprendizaje del ensamblador.	S1-O3 a S1-O5	Documentar cada concepto para su comprensión.
Diferencias entre las computadoras de los miembros del equipo.	S1-O1	Documentar la configuración en dosbox/README.md
Poca disponibilidad para coordinación.	Todos	Coordinación asíncrona mediante GitHub, WhatsApp, Gmail y bitácoras.

Parte B. Reporte de Cierre

Resultado de cada objetivo

Cada objetivo de la Parte A aparece aquí. La evidencia puede ser un commit, una prueba o una captura del depurador.

ID	Estado	Evidencia	Horas reales	Comentarios
S1-O1	Completo	• dosbox/README.md con la configuración. • DOSBox monta la carpeta al abrirse.	1	Incluye la corrección del teclado (F-S1-03).
S1-O2	Completo	Commit 75e3a51 y estructura del repositorio.	0.5	Se resolvieron F-S1-01 y F-S1-02.
S1-O3	Completo	Prueba “Ensamblado”	0.5	/
S1-O4	Completo	Prueba “Modo de Video”	0.5	/
S1-O5	Completo	Pruebas de posición normal y casos límite (figuras 1 y 2).	1	/
S1-O6	Completo	Plan de trabajo entregado.	2	Roles B y C acordados.

Logros de la semana

- El entorno DOSBox con MASM monta la carpeta del repositorio automáticamente al abrirse.
- El repositorio tiene una carpeta por rol, .gitignore y README del laboratorio.
- El programa ensambla, cambia a modo gráfico y dibuja la pelota.
- Se definió la estrategia de migración a Visual Studio.

Pendientes

Ninguno. La semana 2 continúa con el movimiento de pelota, el rebote en paredes, las paletas, el teclado y el inicio del port a Visual Studio.

Reporte de Fallas

ID de la falla y objetivo	F-S1-01
Síntoma observado	Git falla con "Permission denied" al confirmar cambios.
Archivo, rutina y línea	La carpeta .vs/ de Visual Studio no estaba excluida y está bloqueada mientras el programa está abierto.
Categoría	Herramientas
Solución o próximo paso	Se creó .gitignore con la plantilla de Visual Studio.
Estado y responsable	Resuelta : Elihú

ID de la falla y objetivo	F-S1-02
Síntoma observado	Las carpetas docs y dosbox no aparecían en GitHub.
Archivo, rutina y línea	Git no rastrea carpetas vacías.
Categoría	Herramientas
Solución o próximo paso	Se agregaron archivos dentro de cada carpeta.
Estado y responsable	Resuelta : Elihú

ID de la falla y objetivo	F-S1-03
Síntoma observado	En DOSBox los símbolos (-, :) y las mayúsculas salían invertidos.
Archivo, rutina y línea	DOSBox usa por defecto el teclado de Estados Unidos y el estado de Bloq Mayús se sincronizó con Windows.
Categoría	Herramientas

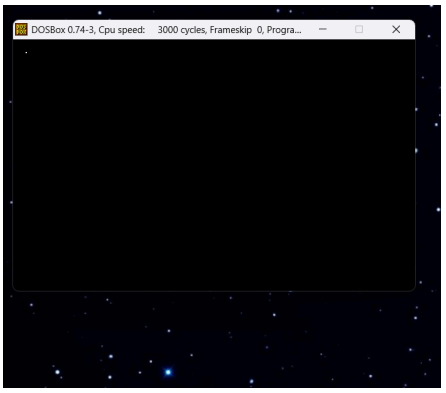
Solución o próximo paso	keyboardlayout=la en la configuración y resincronizar Bloq Mayús.
Estado y responsable	Resuelta : Elihú

Pruebas Ejecutadas

Caso	Rutina	Entrada	Salida esperada	Resultado
Ensamblado	PONG.ASM	Código fuente	Sin errores ni advertencias	Pasa
Modo de video	Inicialización	Ejecutar PONG	Pantalla en modo gráfico con el fondo indicado.	Pasa
Posición normal	DRAW_BALL	Valores iniciales de BALL_X y BALL_Y	Cuadrado visible en esa posición	Pasa
Caso límite 1	DRAW_BALL	BALL_X = 0, BALL_Y = 0	Pelota en la esquina superior izquierda, completa	Pasa
Caso límite 2	DRAW_BALL	Pelota en la esquina inferior derecha	Pelota completa, sin salirse de la pantalla	Por probar

Figura 1

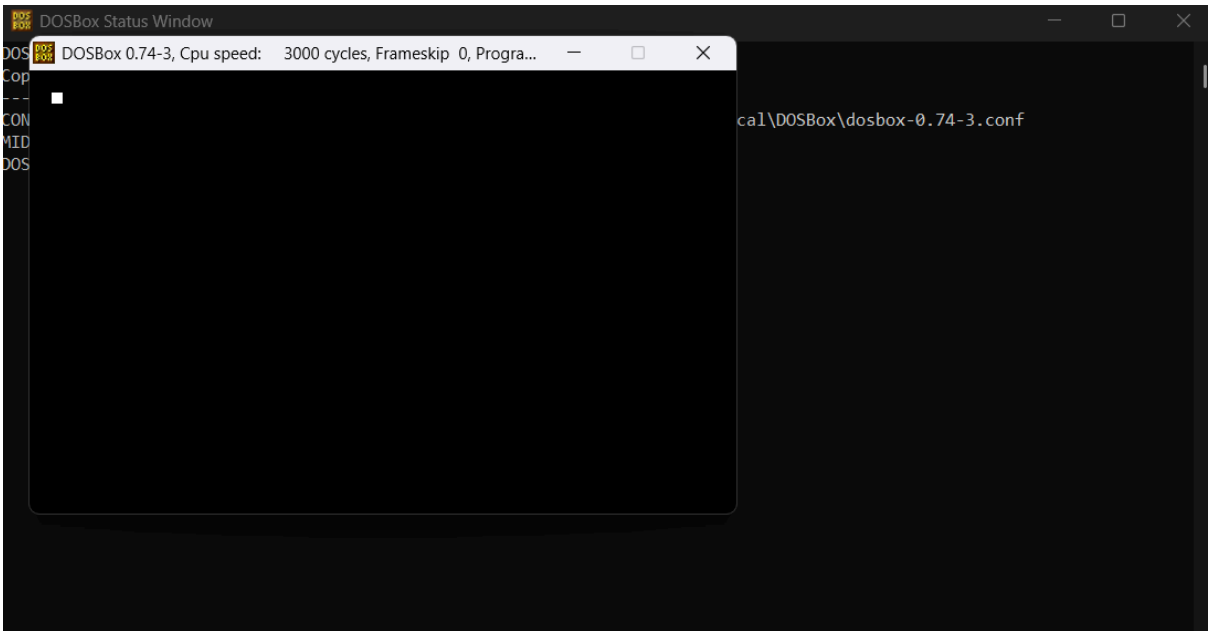
Resultado del Caso Límite 1 en DOSBox



Nota: Elaboración propia.

Figura 2

Resultado del Caso Límite 2 en DOSBox



Nota: Elaboración propia.

Aporte por integrante

Integrante	Trabajo realizado	Horas	Bloqueos o apoyo que necesita
Elihú	Aprendizaje con DOSBox, configuración del entorno, organización del repositorio y elaboración del reporte.	4	/
Leonardo	Acuerdos entre roles y revisión del plan de trabajo.	1	/
Mariana			/

Balance y Ajustes

Objetivos planeados y completados	6 de 6, 100%
Horas estimadas y reales	7 horas aproximadamente.5
Qué funcionó y conviene mantener	● Organizar el repositorio por rol antes de programar. ● Automatizar la configuración de DOSBox.
Qué cambiaremos la próxima semana	● Activar Issues y Milestones en GitHub. ● Que cada integrante registre su bitácora cada semana.
Decisiones técnicas tomadas	● DOSBox se usa como medio de aprendizaje y referencia. El entregable final es la versión en Visual Studio. ● Las interrupciones de video se sustituyen por una pantalla virtual escrita desde __asm. ● Las herramientas de MASM no se suben al repositorio debido a acuerdos de licencia. En su lugar, se documenta el enlace de descarga. ● Los nombres de variables y rutinas se mantienen en inglés.
Pendientes que pasan a la semana 2	Ninguno.